CHAPTER 03

Why LangGraph? From Chains to Graphs

Understand why complex agentic workflows demand a graph-based approach, and how LangGraph solves six fundamental limitations of linear chains.

WHAT THIS CHAPTER COVERS

- LangChain recap & its use cases
- Workflows vs. Agents — key distinction
- Control flow complexity & non-linearity
- Stateful vs. stateless execution
- Event-driven execution & checkpointing
- Fault tolerance: retry & recovery
- Human-in-the-loop patterns
- Nested workflows & subgraphs
- Observability with LangSmith
- When to use LangChain vs. LangGraph

Why LangGraph? From Chains to Graphs

RECAP — CHAPTERS 1 & 2

Chapter 1 introduced the contrast between **Generative AI** (reactive, single-turn) and **Agentic AI** (autonomous, goal-directed, multi-step). Chapter 2 formalised Agentic AI's six characteristics (Autonomy, Goal-Orientation, Planning, Reasoning, Adaptability, Context Awareness) and its five core components (Brain, Orchestrator, Tools, Memory, Supervisor), grounded throughout in an automated HR-recruiter scenario.

1. LangChain — A Quick Recap

What Is LangChain?

LangChain is an *open-source Python library* designed to simplify the development of LLM-based applications. It gives engineers a set of modular building blocks that can be assembled into arbitrarily long processing pipelines without writing low-level orchestration code.

Core Building Blocks

Component	Role	Example
Models	Unified interface to any LLM provider	OpenAI, Anthropic, HuggingFace, Ollama
Prompts	Reusable, parameterised prompt templates	PromptTemplate, ChatPromptTemplate
Retrievers	Fetch relevant documents from a vector store	MMR, similarity search strategies
Chains	Link components so output of one becomes input of the next	Prompt → LLM → OutputParser
Tools	External APIs or Python functions the LLM can call	Weather API, Calculator

What LangChain Is Good At

LangChain excels at *linear workflows* — cases where data flows left-to-right through a fixed sequence of components with no conditional branching, looping, or long-running pauses:

- **Simple chatbots and text summarisers** — user prompt → LLM → formatted response.
- **Multi-step sequential chains** — generate a detailed report, then summarise it, then translate it.
- **RAG applications** — query → retriever → relevant chunks + prompt → LLM → answer.
- **Basic tool-augmented agents** — LLM decides which tool to call; tool returns result; LLM formats response.

IMPORTANT

LangChain's chain abstraction is powerful for **sequential, short-lived workflows**. The moment a workflow becomes non-linear — adding loops, conditional branches, long waits, or human approval steps — LangChain requires large amounts of custom "glue code" outside the library itself, which hurts maintainability.

↻ Quick Revision — LangChain

- LangChain is an open-source library for building LLM-based applications using modular building blocks.
- Core components: Models, Prompts, Retrievers, Chains, Tools.
- Chain = automatic data pipeline where each component's output feeds the next.
- Best suited for linear, short-lived workflows such as chatbots, RAG, and text summarisation.
- Any non-linear logic (loops, conditions) requires custom Python glue code outside LangChain.

2. Workflows vs. Agents — A Critical Distinction

Before diving into LangGraph's advantages, it is essential to understand the difference between an *agentic AI application* (an agent) and a *workflow*. Anthropic's "Building Effective Agents" blog captures this clearly:

Dimension	Workflow	Agent
Who designs the control flow?	The developer, in advance	The LLM, at runtime
Is the execution path fixed?	Yes — static, predetermined	No — dynamically generated each run
Who decides tool usage order?	Hard-coded in code	LLM reasons about it step-by-step
Repeatability	Identical path every run	Path may differ per run
Suitable for	Known, structured business processes	Open-ended, exploratory tasks

The automated hiring flowchart discussed in Chapter 2 is a **workflow** — the developer defines every step and decision in advance. The LLM is used inside individual steps (e.g., drafting a JD, scoring resumes), but the overall orchestration logic is fixed. A true agent would determine those steps itself.

INTERVIEW TIP

Interviewers frequently ask: *"What is the difference between an agent and a workflow?"* Answer: In a workflow, the control-flow path is **predefined by the developer**; in an agent, the LLM **dynamically determines** which steps to take and which tools to call. Both can use LLMs, but agents have more autonomy.

↻ Quick Revision — Workflow vs. Agent

- Workflow: predefined control flow coded by the developer; LLMs used inside steps, not for routing.
- Agent: LLM dynamically decides its own steps, tool usage, and execution order.
- Both exist on a spectrum; real systems often mix workflow structure with agentic decision-making.
- The hiring example is a workflow — complex, but fully pre-designed.

3. Challenge 1 — Control Flow Complexity

Consider the automated hiring workflow: a multi-step process with conditional branches (proceed only if JD is approved), loops (keep modifying the JD until enough applicants arrive), and backward jumps (return to earlier steps on failure). This is a *non-linear workflow*.

Why LangChain Struggles

LangChain's primary abstraction is a **chain** — a linear pipeline. When you need non-linear logic, LangChain provides no built-in constructs for:

- **Conditional branching** — route to step A or step B depending on a runtime condition.
- **Loops** — repeat a step until an exit condition is met.
- **Backward jumps** — return control to an earlier stage of the pipeline.

Developers must implement all of this using ordinary Python (`while` loops, `if/else` statements, `break`/`continue`). This external code — written *around* the library rather than *inside* it — is called *glue code*. The more complex the workflow, the more glue code accumulates, making the codebase harder to maintain, debug, and extend.

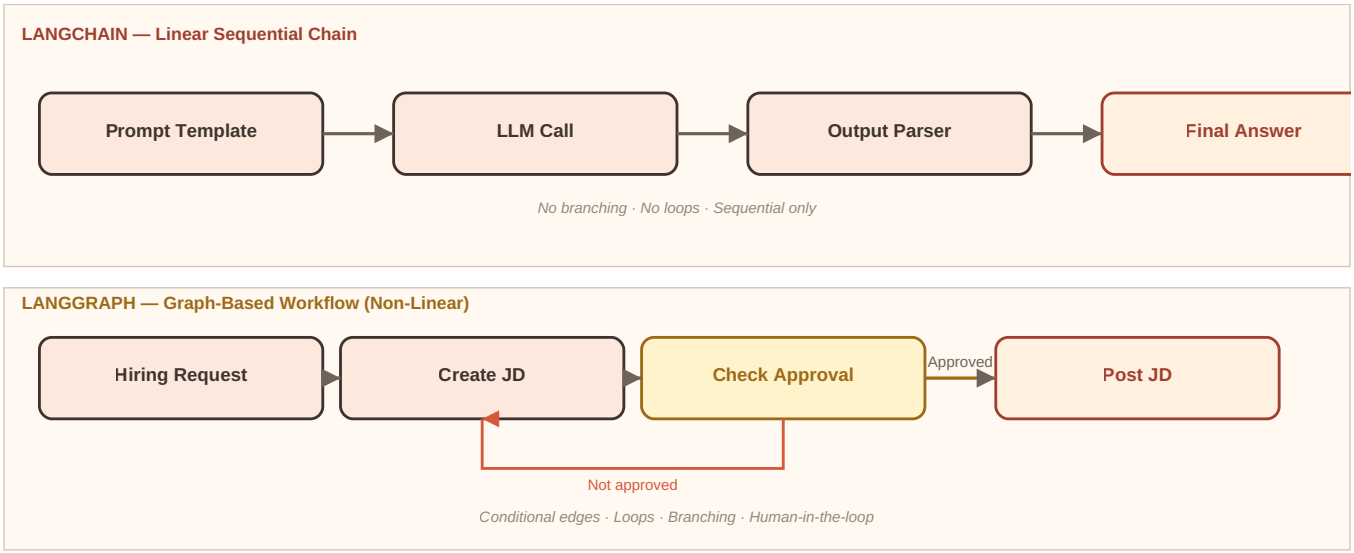

Figure 3.1 — LangChain's linear chain versus LangGraph's graph-based workflow with conditional edges and a loop.

How LangGraph Solves This

LangGraph represents an entire workflow as a *directed graph*, where:

- Every task is a **node** — a Python function.
- Every transition between tasks is an **edge** — which can be unconditional or *conditional*.

Because a graph is inherently non-linear, any combination of branches, loops, and jumps is expressed as native graph structure — no glue code required. Conditional edges automatically route control to different nodes based on a runtime decision function. A loop is simply an edge pointing backward to an earlier node.

> **REAL-WORLD INSIGHT**
>
> In production-grade agentic systems, the volume of glue code is a key indicator of technical debt. LangGraph was specifically designed to eliminate glue code for complex workflows so that **all control flow logic lives inside the graph definition** — making it inspectable, testable, and maintainable.

> ↻ **Quick Revision — Control Flow Complexity**
>
> - Non-linear workflows require conditional branches, loops, and backward jumps.
> - LangChain has no built-in constructs for these — developers must write custom glue code.
> - Glue code = Python logic written outside LangChain to stitch steps together; it hurts maintainability.
> - LangGraph models workflows as directed graphs: nodes = tasks, edges = transitions.
> - Conditional edges in LangGraph route control without any external if/else code.
> - Zero glue code = higher maintainability, easier debugging, cleaner team collaboration.

4. Challenge 2 — State Handling

What Is Workflow State?

Every complex workflow has a set of data points that drive its execution. In the hiring example, these include: the job description text, whether the JD has been approved, whether it has been posted, the number of applications received, the shortlisted candidates, offer letter status, and onboarding status. Collectively, these data points and their current values form the *state* of the workflow.

State is not static — it **evolves** as execution progresses. When the JD is created, `jd_text` is populated. When it is approved, `jd_approved` becomes `True`. The entire workflow depends on reading and updating these values at each step.

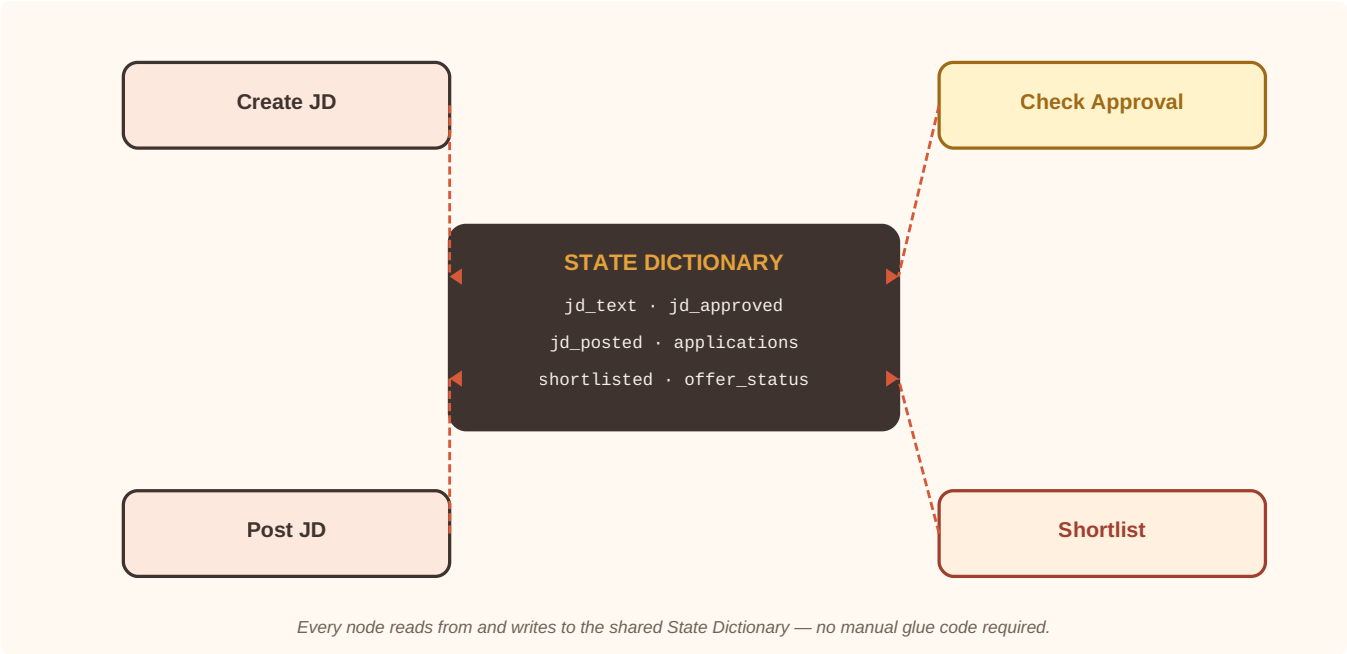

Figure 3.2 — State as a shared, mutable dictionary accessible to every node in the graph.

LangChain — Stateless by Design

LangChain does offer a *conversational memory* concept, but this stores chat history as text — it is not the same as a structured workflow state. For a complex workflow like hiring, LangChain provides no intrinsic mechanism to store and track structured key-value state. Developers must manually maintain a Python dictionary at the top of their script and explicitly update it inside each chain step. This is fragile, error-prone, and scales poorly.

LangGraph — Stateful Execution

LangGraph is built around an explicit *state object*. When you define a graph, you also define a state schema — typically a `TypedDict` or a Pydantic model. LangGraph then:

- Passes the current state into every node as its input.
- Expects each node to return an updated (partial or complete) state as its output.
- Merges the node's output back into the shared state before passing it to the next node.

No node needs to reach into a global variable. State propagation happens automatically and consistently. The entire execution is *stateful* — every node always has access to the latest values of every field.

> **IMPORTANT**
>
> **LangChain is stateless; LangGraph is stateful.** This is one of the most important distinctions in interviews and in practice. LangGraph's stateful execution is the foundation for all its other capabilities: event-driven pausing, fault tolerance, human-in-the-loop, and observability.

↻ Quick Revision — State Handling

- Workflow state = the set of data fields and their evolving values that drive execution.
- LangChain has conversational memory but no structured workflow state tracking.
- Without built-in state, developers manage a manual Python dictionary — fragile and error-prone.
- LangGraph requires an explicit state schema (TypedDict or Pydantic) per graph.
- Every node receives the full current state as input and returns state updates as output.
- LangGraph merges updates automatically — no manual state management required.
- Stateful execution is the foundation for checkpointing, fault tolerance, and HITL.

5. Challenge 3 — Event-Driven Execution

Sequential vs. Event-Driven

Workflows can execute in two fundamentally different modes:

- **Sequential execution:** Once started, the workflow runs from beginning to end without stopping. Each step triggers the next immediately upon completion. LangChain chains work this way.
- **Event-driven execution:** The workflow *pauses* at certain points and waits for an external trigger before resuming. The trigger might be the passage of time, a human approval, a candidate's reply, or a webhook.

Why the Hiring Workflow Needs Event-Driven Execution

The hiring workflow has several natural pause points:

- After posting the JD — wait **7 days** for applications.
- After modifying the JD — wait **48 hours** to monitor response.
- After sending an offer letter — wait for the **candidate to accept or reject**.

These are not brief computational waits. They may last hours, days, or weeks. A LangChain chain has no ability to pause in the middle of execution and resume later — it assumes it will run to completion without interruption.

LangGraph — Checkpointing

LangGraph solves this with its *checkpointer* mechanism. Since every node execution updates the state and LangGraph tracks that state continuously, you can:

1. Pause execution at any node.
2. Save the current state to memory or an external database via a checkpointer.
3. Shut down the process entirely (freeing compute resources).
4. When the external trigger arrives, reload the saved state and resume execution from exactly where it paused — not from the beginning.

REAL-WORLD INSIGHT

Think of LangGraph's checkpointing like a video game save system. You can save your progress at any point, exit the game, and resume from exactly that stage the next day. LangGraph provides the same capability for long-running agentic workflows that span hours or days.

↻ Quick Revision — Event-Driven Execution

- Sequential execution: workflow runs start-to-finish without pause (LangChain chains).
- Event-driven execution: workflow pauses, waits for an external trigger, then resumes.
- LangChain is built for sequential execution only — no built-in pause/resume capability.
- LangGraph uses a checkpointer to save state at any node and resume from that exact point.
- Checkpointers can store state in-memory or in external databases (e.g., SQLite, Redis).
- Event-driven execution is essential for long-running workflows with human or time-based triggers.

6. Challenge 4 — Fault Tolerance

Why Fault Tolerance Matters

A workflow that runs for days or weeks is statistically likely to encounter failures. Two categories of fault exist:

- **Node-level faults** (small): A single step fails — e.g., the LinkedIn API is temporarily unavailable when posting the JD.
- **System-level faults** (large): The server or container running the workflow crashes, losing all in-memory state.

A *fault-tolerant* system recovers from both categories without requiring the user to restart the entire workflow from scratch.

LangChain — No Fault Tolerance

LangChain assumes its chains are *short-lived* — triggered, executed, and done within seconds or minutes. There is no built-in retry mechanism for failed steps and no concept of resuming a chain from a mid-point after a crash. If a 5-step chain fails on step 3, you must re-run from step 1.

LangGraph — Built-in Retry and Recovery

LangGraph addresses both fault categories explicitly:

RETRY (NODE-LEVEL FAULTS)

Nodes can be configured with retry logic. If a node raises an exception (e.g., an API timeout), LangGraph can automatically wait and re-attempt the same node before propagating the error. This handles transient failures like rate limits, network blips, and temporary service outages.

RECOVERY (SYSTEM-LEVEL FAULTS)

Because LangGraph checkpoints the state after every node execution, a system crash merely means the most recent checkpoint is the last known good state. You can call `graph.invoke()` with the saved state and LangGraph will identify which node was in progress, skip all previously completed nodes, and resume from the correct point.

> **BEST PRACTICE**
>
> For production deployments of long-running LangGraph workflows, always configure an **external checkpointer** (e.g., a SQLite or PostgreSQL-backed checkpointer) rather than in-memory storage. This ensures state survives process restarts, container redeployments, and infrastructure events.

↻ Quick Revision — Fault Tolerance

- Node-level fault: a single step fails (e.g., API down) — handle with retry logic.
- System-level fault: the entire server/container crashes — handle with recovery from checkpoint.
- LangChain has no built-in fault tolerance; a crashed chain must restart from step 1.
- LangGraph checkpoints state after every node; on crash, resume from the last checkpoint.
- Retry config in LangGraph nodes handles transient errors (API timeouts, rate limits).
- Use an external (persistent) checkpointer in production to survive infrastructure events.

7. Challenge 5 — Human-in-the-Loop

What Is Human-in-the-Loop?

Many real-world workflows require a human to review, approve, or provide input before execution can continue. Examples in the hiring scenario: a manager must approve the JD before it is posted, HR must confirm which candidates to shortlist, and a candidate must accept or reject an offer. The pattern of *pausing execution to await human input* is called *Human-in-the-Loop (HITL)*.

Why LangChain Cannot Do This Well

A synchronous LangChain chain can technically prompt a user for input mid-execution. However, since the chain runs in a blocking, sequential fashion, the process must remain alive — consuming compute resources — while waiting. If a manager approval takes 24 hours, the script must run (and potentially crash) for 24 hours. Splitting the workflow into two separate chains solves the blocking problem but forces the developer to manually serialize and transfer state between chains — more glue code.

LangGraph — HITL as a First-Class Feature

LangGraph explicitly supports HITL. From its documentation: *"LangGraph allows you to pause execution indefinitely — for minutes, hours, or even days — until human input is received. This is possible because LangGraph checkpoints the graph state after every step, which allows the system to persist execution context and later resume the workflow continuing from where it left off. This supports asynchronous human review or input without time constraints."*

The mechanism is the same checkpointer used for fault tolerance: save state, pause, wait for human input (which arrives via a separate API call or UI event), reload state, and resume. No compute is consumed during the wait.

> **INTERVIEW TIP**
>
> When asked about HITL in agentic systems, explain that it requires two capabilities: (1) the ability to **pause and persist** execution state, and (2) the ability to **resume from an arbitrary checkpoint** after receiving external input. LangGraph provides both through its stateful checkpointing architecture. LangChain does not.

> ↻ **Quick Revision — Human-in-the-Loop**
>
> • HITL = pausing a workflow at a defined point to await human review, approval, or input.
>
> • Common HITL scenarios: JD approval, shortlisting confirmation, offer negotiation.
>
> • LangChain can only block a running process waiting for input — not suitable for long waits.
>
> • LangGraph pauses at any node, saves state via checkpointer, and resumes asynchronously.
>
> • No compute is consumed during a HITL pause — the process can shut down and restart.
>
> • HITL is a first-class, explicitly documented feature in LangGraph.

8. Feature — Nested Workflows and Subgraphs

In LangGraph, it is possible to use an entire compiled graph as a single *node* inside a larger graph. This concept is called a *subgraph*. The official definition: *"A subgraph is a graph that is used as a node in another graph."*

Why Subgraphs Are Useful

Consider the "Conduct Interviews" step in the hiring workflow. At the surface level it appears to be a single node, but it is itself a complex multi-step process: generate candidate-specific questions, send reminders, run round 1, evaluate, run round 2, evaluate, run round 3, produce final recommendation. This internal process has its own state, its own loops, and its own conditional branches — it deserves its own graph.

Subgraphs enable two major capabilities:

1. MULTI-AGENT SYSTEMS

Each agent in a multi-agent system can be implemented as its own graph. A supervisor agent (the outer graph) delegates to specialised agents (inner subgraphs). Example: a self-driving car AI might have a sensor-processing agent, a driving-control agent, and an entertainment agent — each a separate graph — coordinated by a top-level orchestrator graph.

2. REUSABILITY

Just as functions in programming promote reuse, subgraphs make workflow components reusable. An approval workflow graph can be designed once and used at every point in the hiring flow where approval is required — JD approval, posting approval, offer approval — without duplicating code.

REAL-WORLD INSIGHT

Multi-agent architectures are increasingly common in production agentic systems. LangGraph's subgraph capability is the primary mechanism for building them — each agent is a graph, and the coordination layer is another graph on top. This maps directly to how complex human organisations delegate work through hierarchies.

↻ **Quick Revision — Nested Workflows / Subgraphs**

- Subgraph: a compiled LangGraph graph used as a node inside another graph.
- Enables nested workflows — a complex step can have its own internal graph structure.
- Use case 1: Multi-agent systems — each agent is a subgraph; a supervisor coordinates them.
- Use case 2: Reusability — build a workflow once (e.g., approval flow) and reuse it anywhere.
- Subgraphs have their own state; the parent graph and subgraph share state via defined interfaces.
- Not natively possible in LangChain — nested workflows require custom orchestration.

9. Challenge 6 — Observability

What Is Observability?

Observability refers to how easily you can *monitor, debug, and audit* what your workflow is doing at runtime. When an agentic system is deployed in production, it is critical to know: which node executed when, what state it received, what it produced, which LLM prompts were sent, and how long each step took. This is needed for debugging unexpected behaviour, auditing AI decisions, and optimising costs.

LangSmith — The Observability Layer

LangSmith is Anthropic's (LangChain team's) dedicated observability platform for LLM applications. It records every LLM invocation: the prompt sent, the response received, token counts, and latency. When integrated with LangChain, it traces chain-level activity.

The LangChain Gap

The fundamental problem is that LangSmith can only trace *LangChain code*. It cannot observe the custom glue code written in plain Python outside the library. In a complex LangChain-based workflow with significant glue code, you get only *partial observability* — you see what the LLMs did, but not the surrounding control logic. A loop's iteration number, for example, is invisible to LangSmith if the loop is a plain Python `while` statement.

LangGraph — Full Observability

Because all control flow in LangGraph lives inside the graph definition (no external glue code), LangGraph can report the complete execution timeline to LangSmith:

- Every node that executed, in order.

- The state before and after each node.
- All LLM calls with prompts, responses, token counts, and latency.
- Every human-in-the-loop pause and the moment human input was received.
- State changes introduced by each update.

This produces a full chronological audit trail from workflow start to completion — invaluable for debugging, compliance, and cost analysis.

COMMON MISTAKE

Developers sometimes assume that integrating LangSmith is sufficient for full observability of a complex LangChain-based workflow. It is not — LangSmith only traces the LangChain portions. If significant control flow logic exists outside LangChain (glue code), those portions produce **no trace data**, creating blind spots in your observability.

↻ **Quick Revision — Observability**

- Observability = ability to monitor, debug, and audit a workflow's runtime behaviour.
- LangSmith is the LangChain team's observability platform — records LLM calls, prompts, tokens, latency.
- LangSmith can only trace LangChain code — it cannot observe custom Python glue code.
- Complex LangChain workflows produce only partial observability due to glue code blind spots.
- LangGraph keeps all control flow inside the graph, so LangSmith receives complete traces.
- Full observability includes: node execution order, state before/after each node, all LLM calls, HITL events.

10. LangGraph — Introduction and When to Use It

What Is LangGraph?

LangGraph is an orchestration framework built on top of LangChain that enables you to build **stateful, multi-step, event-driven workflows** using LLMs. It was created by the LangChain team specifically to address the limitations of linear chains when building complex agentic AI applications.

The core idea: model your entire workflow as a *directed graph*. Define each task as a node (a Python function). Define each transition as an edge (unconditional or conditional). LangGraph then handles state management, conditional routing, looping, checkpointing, pause/resume, retry, and recovery — without requiring any glue code.

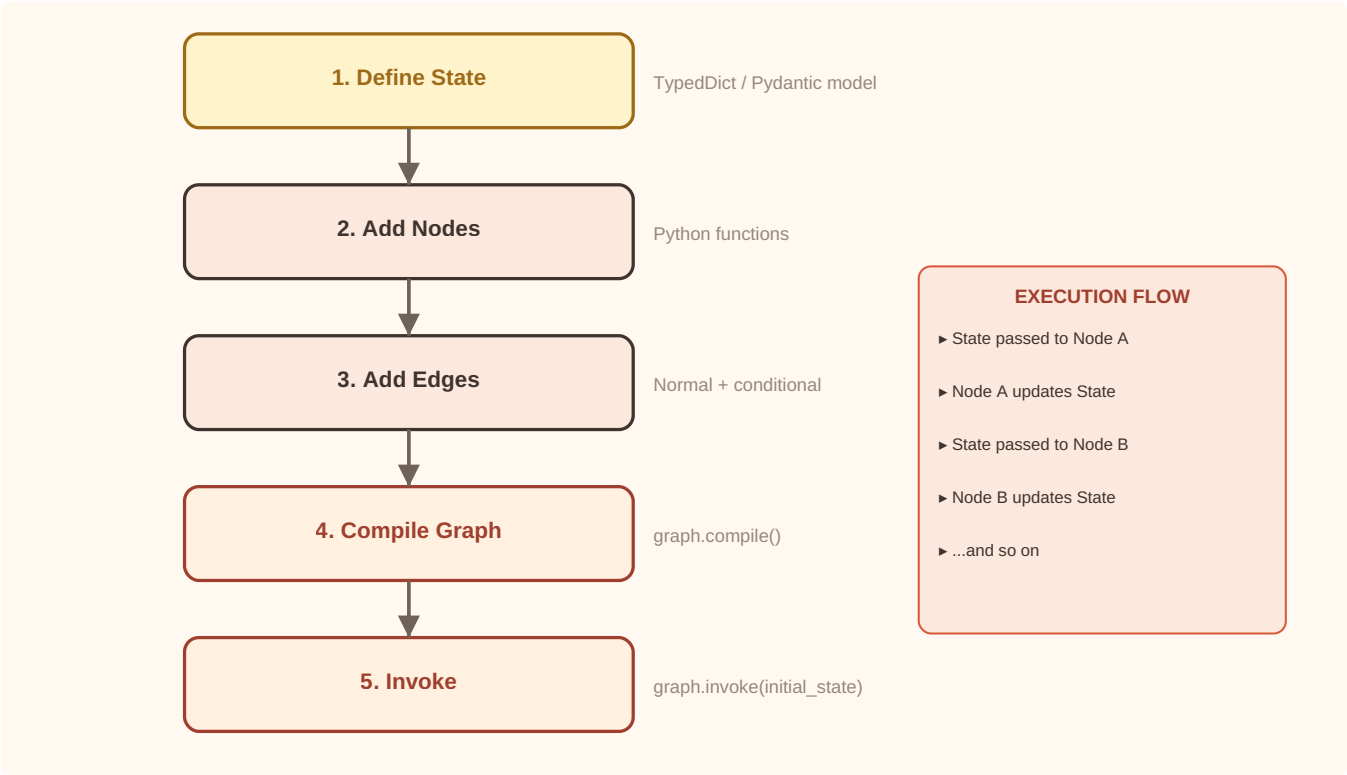

Figure 3.3 — LangGraph execution model: the five build steps and the stateful node-by-node execution flow.

Core Conceptual Model

LangGraph Concept	Maps To	In the Hiring Example
Node	A Python function; one task	create_jd(), check_approval(), post_jd()
Edge	Unconditional transition to the next node	Hiring Request → Create JD
Conditional Edge	Routes to different nodes based on a function	Approved? → Post JD or → Create JD
State	Shared TypedDict/Pydantic object	jd_text, jd_approved, applications_count…
Checkpointer	State persistence layer	Saves progress; enables pause, resume, recovery
Subgraph	A graph used as a node	Conduct Interviews as its own internal graph

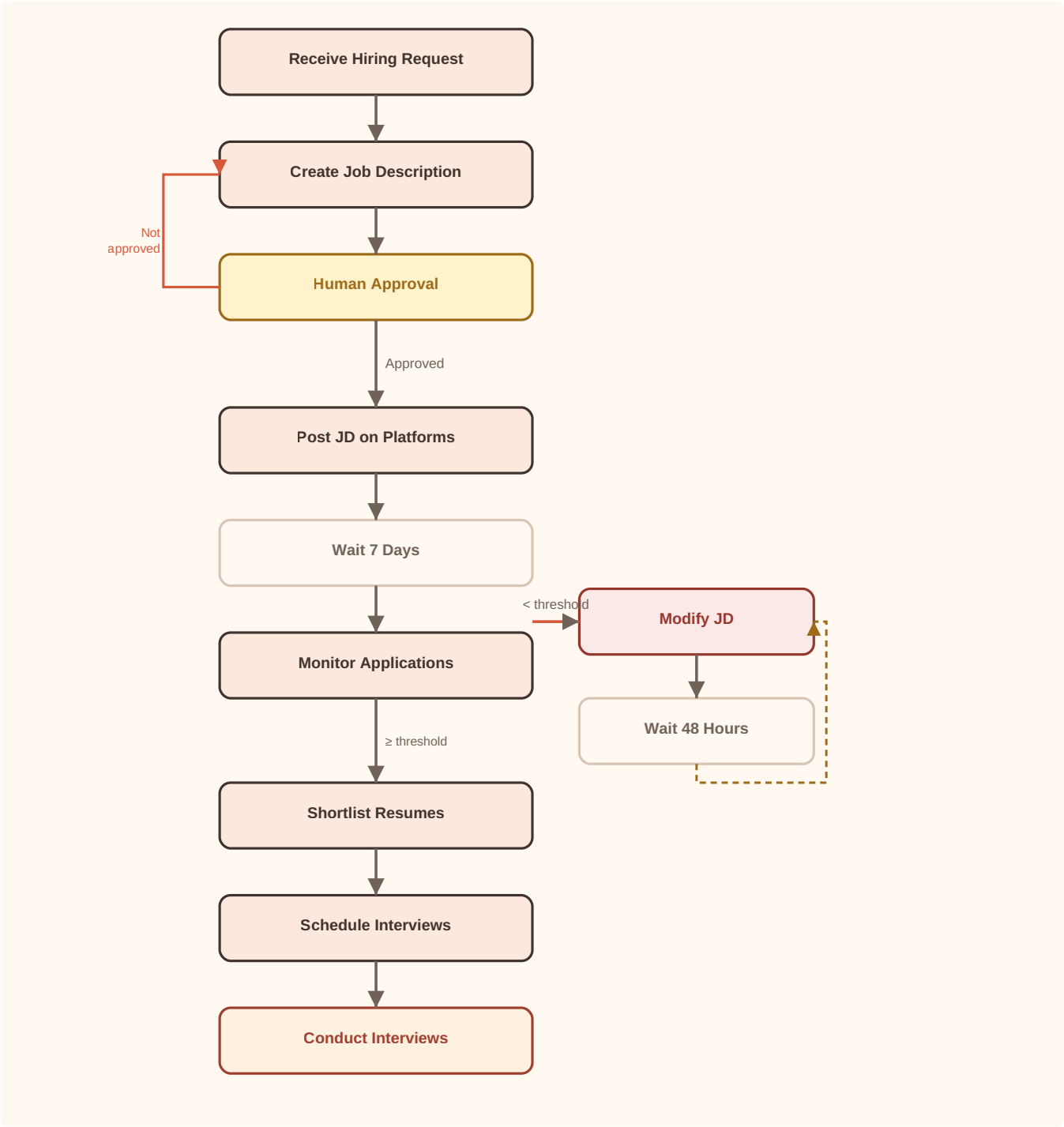

Figure 3.4 — The automated hiring workflow as a LangGraph-style directed graph with conditional edges, loops, and tool integrations.

LangChain vs. LangGraph — When to Use What

Scenario	Use LangChain	Use LangGraph
Simple chatbot or text summariser	✓	
Basic RAG application	✓	
Short, sequential multi-step pipeline	✓	
Complex non-linear workflow with loops		✓
Conditional branching at runtime		✓
Long-running workflow (hours/days)		✓
Human approval / HITL required		✓
Multi-agent coordination		✓
Full runtime observability needed		✓
Fault-tolerant, production-grade system		✓

IMPORTANT

LangGraph is **built on top of LangChain** — it does not replace it. All the LangChain components you already know (ChatOpenAI, PromptTemplate, retrievers, tools, document loaders, text splitters) are still used inside LangGraph nodes. LangGraph adds the orchestration layer; LangChain provides the components. They work together in every LangGraph application.

↻ Quick Revision — LangGraph

- LangGraph is an orchestration framework built on top of LangChain for stateful, graph-based workflows.
- Workflow = directed graph: nodes (Python functions) + edges (transitions) + conditional edges.
- LangGraph handles state, routing, loops, checkpointing, pause/resume, retry, and recovery.
- LangChain components (LLMs, prompts, retrievers, tools) are used inside LangGraph nodes.
- Use LangChain alone for simple, linear, short-lived workflows.
- Use LangGraph for complex, non-linear, long-running, or multi-agent workflows.
- LangGraph replaces LangChain's chain orchestration — not LangChain's building blocks.

K. Key Takeaways

- **LangChain** provides modular building blocks (Models, Prompts, Retrievers, Chains, Tools) that are excellent for linear, short-lived LLM workflows such as chatbots and RAG applications.
- **Workflows vs. Agents:** In a workflow the developer pre-defines the control flow; in an agent the LLM dynamically determines its own steps and tool usage.
- **Control Flow Complexity:** LangChain has no built-in constructs for conditional branching, loops, or jumps — developers must write glue code, which hurts maintainability. LangGraph expresses all of this natively as graph edges.
- **State Handling:** LangChain is stateless; LangGraph is stateful. LangGraph passes a shared TypedDict/ Pydantic state object through every node, which any node can read or update — no manual dictionary management required.
- **Event-Driven Execution:** LangChain chains run sequentially to completion. LangGraph can pause at any node, save state via a checkpointer, and resume when an external trigger (time, human, API) fires.
- **Fault Tolerance:** LangGraph provides built-in retry for node-level faults and recovery from the last checkpoint after system-level crashes. LangChain has neither.
- **Human-in-the-Loop:** LangGraph can pause indefinitely awaiting human input without consuming compute resources — a first-class, documented feature. LangChain can only block a running process.
- **Subgraphs / Nested Workflows:** LangGraph allows graphs-within-graphs, enabling multi-agent system design and reusable workflow components. Not possible in LangChain.
- **Observability:** LangGraph integrates tightly with LangSmith and provides a complete execution timeline because all control flow lives inside the graph. LangChain gives only partial observability due to glue code blind spots.
- **Complementary, not competing:** LangGraph is built on LangChain. LangChain components power individual nodes; LangGraph orchestrates the graph. Both are used together in every real LangGraph application.

R. Revision Sheet

LangChain

Open-source library for LLM apps. Modular building blocks (Models, Prompts, Retrievers, Chains, Tools) for linear, short-lived workflows.

Workflow vs. Agent

Workflow: developer-defined, static control flow. Agent: LLM dynamically determines steps and tool calls at runtime.

Glue Code

Custom Python logic written *outside* LangChain to implement loops, conditions, and jumps. Accumulates with complexity; hurts maintainability.

LangGraph

Orchestration framework on top of LangChain. Represents workflows as directed graphs: nodes = Python functions, edges = transitions.

State Object

TypedDict or Pydantic model shared across all nodes. Every node receives current state as input and returns state updates as output.

Conditional Edge

An edge in LangGraph that routes execution to different nodes based on the return value of a routing function.

Checkpointer

LangGraph mechanism to save the full state snapshot after every node. Enables pause/resume, fault recovery, and HITL.

Event-Driven Execution

Workflow pauses at a node and waits for an external trigger (time elapsed, human input, API event) before resuming.

Fault Tolerance

Retry = re-attempt a failed node. Recovery = resume from the last checkpoint after a system crash. Both are built into LangGraph.

Human-in-the-Loop (HITL)

Pausing a workflow indefinitely for human review/ approval. LangGraph checkpoints state; resumes asynchronously after input arrives.

Subgraph

A compiled LangGraph graph used as a node inside a parent graph. Enables multi-agent coordination and reusable workflow components.

LangSmith

Observability platform for LLM applications. LangGraph integrates tightly with it, providing complete node-level execution traces — unlike partial LangChain traces.

I. Interview Questions

Q1. What is the difference between a workflow and an agent in the context of agentic AI?

Q2. What are the key limitations of LangChain when building complex, non-linear workflows?

Q3. What is “glue code” and why is it a problem in complex LangChain applications?

Q4. How does LangGraph represent a workflow, and what are its core building blocks?

Q5. What is the difference between stateless and stateful workflow execution? Which does LangGraph support and how?

Q6. What is a checkpointer in LangGraph and what problems does it solve?

Q7. Explain the concept of Human-in-the-Loop (HITL). How does LangGraph implement it?

Q8. What are LangGraph subgraphs? Name two use cases for them.

Q9. Why does LangChain provide only partial observability for complex applications, and how does LangGraph improve on this?

Q10. If you were starting a new project that requires conditional routing, 7-day wait periods, and human approval steps, would you choose LangChain or LangGraph? Justify your answer.

F. Further Reading

- **LangGraph Official Documentation** — langgraph.dev: covers StateGraph, nodes, edges, checkpointers, HITL, and subgraphs with code examples.
- **Anthropic Blog — “Building Effective Agents”** — anthropic.com/research/building-effective-agents: the canonical reference for the workflow vs. agent distinction.
- **LangSmith Documentation** — smith.langchain.com/docs: covers integration with LangGraph, trace visualisation, and production monitoring.
- **LangGraph GitHub Repository** — github.com/langchain-ai/langgraph: source code, examples, and pre-built checkpointer implementations (MemorySaver, SQLiteSaver).
- **LangChain LCEL Documentation** — python.langchain.com/docs/expression_language: LangChain Expression Language — the modern way to compose LangChain chains that feeds into LangGraph node functions.